

# Resource Contention in Shared-Memory Multiprocessors: A Parameterized Performance Degradation Model\*

ARUN K. NANDA, HONDA SHING, TEN-HWAN TZEN, AND LIONEL M. NI

Department of Computer Science, Michigan State University, East Lansing, Michigan 48824-1027

A standard metric conventionally employed to compare the performance of different multiprocessor systems is *speedup*. Although providing a measure of the improvement in execution speed achievable on a system, this metric does not yield any insight into the factors responsible for limiting the potential improvement in speed. This paper studies the performance degradation in shared-memory multiprocessors as a result of contention for shared-memory resources. A *replicate workload framework* with a flexible mechanism for workload specification is proposed for measuring performance. Two normalized performance metrics—*efficiency* and *overhead factor*—are introduced to quantify the factors limiting performance and facilitate comparison across architectures. Finally, the proposed model is employed to measure and compare the performance of three contemporary shared-memory systems, with special emphasis on the newly released BBN Butterfly-II (TC2000), currently undergoing Beta test. © 1991 Academic Press, Inc.

## 1. INTRODUCTION

Within the last decade, the design and analysis of multiprocessor systems has drawn a considerable amount of attention from the research community. The primary attraction motivating this interest is the tremendous potential that exists in multiprocessor systems for speeding up applications. The usual metric employed to characterize this gain in speed is the ratio of the execution time of the sequential version of a program in a uniprocessing environment to the running time of its parallel counterpart in a multiprocessing environment (say, with  $n$  processors). This ratio yields a dimensionless quantity that can be interpreted as the “speedup.”

The actual speedup achieved on a given multiprocessor system depends on the nature of the application being run. Ideally, if the application exhibits a high degree of inherent parallelism such that different functional/data components of the application can be executed in parallel on  $n$  processors totally independent of each other, then it is possible to attain a perfect speedup factor of  $n$  (called *ideal speedup*). However, in practice, there are several factors that may prevent one

from attaining such an ideal speedup. First, most applications contain code segments that are inherently sequential in nature and hence must be run in a serial fashion. Second, the application segments that run in parallel on different processors, as *parallel processes*, often must interact with each other either to exchange data or to synchronize their executions. These process interactions not only cause the parallel processes to mutually influence the speed of execution of their peers, but also introduce an overhead component into the total runtime of the application resulting from communication latencies. Third, a number of hardware resources such as shared memory, communication channels, and I/O channels are shared among the processes executing in parallel. Each such shared resource can be accessed by only a single process at a time in a mutually exclusive fashion. Thus, contention for shared resources and the consequent queuing delay introduces yet another overhead component into the total runtime of the application.

Shared-memory multiprocessors constitute an important class of parallel processing systems to which a large proportion of the currently popular multiprocessors belong. The architecture of machines representing this class is characterized by a number of processing elements connected to a comparable number of memory elements through an interconnection network. The interconnection medium selected governs the communication bandwidth of the system and hence determines the maximum speed at which processes running on different processors can interact. The communication medium used in the currently available architectures [9] includes crossbar switches (systems with <10 processors), high-speed bus (systems with up to 30 processors), and Multistage Interconnection Networks or MINs (systems with up to a few hundred processors). Since all accesses to memory are arbitrated through the communication medium, the bandwidth and the arbitration protocol of the communication medium are important determinants of efficiency in the execution of a parallel program.

When multiple concurrent processes belonging to a parallel application run on a shared-memory multiprocessor, they interact through globally shared-data structures. Exclusive access to such shared-data structures is enforced by their encapsulation within *critical sections*. Entry into these critical

\* This work was supported in part by NSF Grant ECS-88-14027.

sections is controlled through the use of *locks* either directly in the application program or indirectly in a subroutine library or the operating system. Given sufficiently high rates of locking, the chief factor preventing linear speedup (besides lack of adequate inherent parallelism in the application) is lock contention—the blocking of processes that are trying to acquire a lock currently held by another process. Further, since all processes must access the same shared-lock variables, contention arises at the hardware level in the communication channel as well as at the shared memory. Depending on the arbitration strategy, all conflicting processes except one must back out of the contention and hence incur a delay, again preventing linear speedup. Thus, it is apparent that when multiple processes execute concurrently, cooperating with one another, the overall speedup is very likely to fall short of the ideal linear speedup. This loss in performance can be attributed to the various kinds of parallel execution overheads described above.

There have been several studies which analyze the performance of multiprocessor systems. Most of these employ either analytical models [11, 17] or simulation-based techniques [1, 12]. Those which measure actual systems are usually designed to derive measures such as MFLOPS or MIPS and are not concerned with degradation in performance due to system overhead and resource contention. In [7], system overhead is analyzed when the performance of the Alliant FX/8 is modeled. However, very little work done to date actually quantifies, with real measured data, the degradation of multiprocessor performance and the different components of overhead associated with running a parallel program. This paper addresses the problem of measuring and analyzing the loss in performance of concurrent processes running on shared-memory multiprocessors. Moreover, the experimental results presented in this paper focus on the newly released BBN Butterfly-II (TC2000) system running release 2 Beta, for which no performance indices are available to date.

Instead of measuring the possible speedup achievable in the parallel execution of typical applications, we make an attempt to do something just the opposite. The loss in performance resulting from the execution of multiple identical processes contending for shared-memory resources is studied and quantified. The total measured overhead, accounting for the loss in efficiency, is then apportioned into the associated components. First, a replicate workload framework is proposed to study the performance degradation in concurrent program execution. Next, the proposed model is employed to measure and compare the performance of three contemporary shared-memory multiprocessors, namely the BBN Butterfly-I (GP1000), the BBN Butterfly-II (TC2000), and the Sequent Balance 21000. The framework includes

- a flexible parameterized mechanism for representing a wide range of artificial program workloads,

- a workload emulation program kernel, and
- an analytical formulation to predict and interpret the relationship between a specified workload and the expected performance.

The rest of this paper is organized as follows. Section 2 provides a brief background of the shared-memory architectures considered in this paper and describes the scope and objective of the performance measurements performed. Section 3 elaborates the framework established for the measurement so that fair comparisons can be made across architectures. It explains the input parameters, the output metrics, the structure of the program used, and the underlying motivation behind the design choices made. In Section 4, a mathematical model is formulated to aid in the interpretation of performance loss. The values obtained from actual measurements are presented in Section 5 and analyzed to offer viable explanations for the observed behavior in the context of the target architecture. Finally, Section 6 summarizes and concludes the main results obtained.

## 2. PERFORMANCE MEASUREMENT OBJECTIVE

The principal systems used in our investigation were the 96-node BBN GP1000 at Michigan State University, the 32-node BBN TC2000, and the 24-node Sequent Balance 21000 at Argonne National Laboratory. These three systems are, henceforth, referred to as the BBN-1, the BBN-2, and the Balance, respectively. In this section, the salient features of the processor-memory interconnection architecture of these shared-memory systems are briefly reviewed only in as much as they facilitate the interpretation of the results obtained. Also, the nature and characteristics of the different kinds of overheads arising in the execution of concurrent processes contending for memory are examined.

### 2.1. Shared-Memory Architectures

The BBN systems are physically distributed-memory multiprocessors running the MACH operating system that offer a logically shared virtual space to the programmer [19, 20]. This is made possible by a specially designed MIN, a slight variant of the Omega network, that allows any processing element to access any memory module. Each processor node in the system consists of a processing element, a local memory module (that can also be accessed by other nodes), and a network interface controller that arbitrates and schedules all remote access requests through the interconnection network. The MIN can be visualized as a cylindrical switching network connecting the nodal processing elements on the one end to the nodal memory modules at the other end. A local memory reference directly accesses the memory, bypassing the interconnection network, whereas a remote reference is serviced through the network. In the

event of conflicts at an intermediate switch on the network, the entire requesting path and hence the request is dropped and retried after an interval of time. There is no local caching of data references and hence no need for coherence maintenance protocols.

On the other hand, the Balance is a truly shared-memory multiprocessor running the DYNIX (a variant of UNIX) operating system [18]. The system is built around a high-bandwidth bus to which all the processing elements, memory modules, and I/O interfaces are attached. All memory references are serviced through the bus. Multiple requests are pipelined onto the bus so that the bus is available for reuse while a memory access request is being serviced. Thus a processor does not hold the bus for the entire duration of a memory access. This significantly improves the bus utilization and bandwidth. Access to hardware locks is arbitrated over a separate dedicated bus. Each processor has an on-board data cache that is governed by a Write-Through policy. Coherence is enforced by "snoopy" cache controllers that watch and monitor the bus continuously.

The BBN-2 is constructed using the Motorola 88100 processor [20], which is based on the reduced instruction set computer (RISC) architecture, as opposed to the Motorola 68020 processors used in the BBN-1, which are of the complex instruction set (CISC) class. The RISC processors employ a very simple set of instructions that are optimized for speed and are implemented in hardware, rather than using microcode. By focusing on register-to-register operations and limiting memory-based operations to separate load and store instructions, the RISC processors achieve extremely fast performance. The Butterfly switch (MIN) is the key component that provides the BBN systems with scalable bandwidth capable of supporting multiple memory accesses at the same time. Each switch module in the 2-stage MIN of BBN-2 is an  $8 \times 8$  crossbar as compared to the 4-stage MIN composed of  $4 \times 4$  crossbars in the BBN-1.

The architectural features of the three systems to which we had access are summarized in Table I.

## 2.2. Overheads from Hardware

In the BBN systems, all remote memory accesses have to go through the interconnection network whereas local memory references do not [19, 20]. Consequently, remote mem-

ory references incur the additional cost of communication latency over the network from a standpoint of speed. In this sense, access to all memory is not uniform on the BBN systems. In terms of time consumed, remote accesses are deemed to be more expensive. This situation is significantly different from that of a truly shared-memory system like the Balance, wherein access to all memory locations is uniform in terms of speed [18]. There is no distinction between local and remote memory modules and hence no extra latency incurred by remote accesses.

With the nonuniform organization of memory on the BBN systems, the network latency component of the memory access time can be a significant hindrance to good performance. Since a shared-data item located in a given memory module is remote to all executing processes except one, an overwhelming proportion of the shared-data accesses exhibit the worst-case latency. Hence, the distribution of shared-data items across the processor nodes, so as to optimize the total access time of the memory reference pattern generated by a parallel program, is not a trivial task. Quantification of the latency overhead component, and its relative contribution to the total overhead leading to performance degradation, yields useful information that helps in deciding on the distribution of data to the participating nodes. We have termed this overhead component *latency* in the performance metrics.

Another significant delay that arises from pure hardware considerations is due to contention on the interconnection network and at the memory modules. When multiple processes running on different processors attempt to access the same memory module simultaneously (not necessarily in a mutually exclusive fashion), the memory hardware can service only one of the requests (assuming multiport memories are not used), forcing the others to wait for service. Thus simultaneous access requests to a shared-data location result in contention and are inherently serialized by the hardware. A similar hardware contention also arises at the bus (in Balance) or at the switches in the interconnection network (in BBNs) if the processor-to-memory paths of two different references overlap. Then, as mentioned earlier, the request that encounters the blocked path is withdrawn and resubmitted later. In the case of the BBN systems, alternate paths may be found through the network depending on the exact configuration of the system, but that does not change the validity of our discussion. We represent both these hardware con-

TABLE I  
Summary of System Architectural Features

| System  | Processor type | No. of processors | Data cache | Interconnection network  | Bandwidth per channel |
|---------|----------------|-------------------|------------|--------------------------|-----------------------|
| BBN-1   | MC68020        | 96                | No         | 4-stage $4 \times 4$ MIN | 32.0 Mbits/s          |
| BBN-2   | MC88100        | 32                | No         | 2-stage $8 \times 8$ MIN | 38.0 Mbytes/s         |
| Balance | NS32000        | 24                | Yes        | Single bus               | 26.7 Mbytes/s         |

tention overheads by a composite quantity called *contention* in the performance metrics.

### 2.3. Overheads from Software

All shared-memory multiprocessors provide some form of hardware support for making mutually exclusive access to shared-data structures. This support usually consists of instructions that atomically read and then write a single memory location. Atomic instructions serve two purposes. First, if the operations on the shared data are simple enough, they can be encapsulated into single atomic instructions. Mutual exclusion is directly guaranteed in hardware. If a number of processors simultaneously attempt to update the same location, each waits its turn without returning control back to software. Second, for critical sections that take more than one instruction, a software protocol is needed. The protocol specifies how to acquire and release a lock guarding the critical section. Provided that each processor obeys the protocol, mutual exclusion is guaranteed for the code between the acquisition and the release of the lock. Atomic instructions are used to arbitrate between simultaneous attempts to acquire the lock. If the lock is busy, then spin waiting is usually done in software to wait for the lock to be freed. Unfortunately, naive implementations of spin waiting, where a globally shared-lock location is accessed continuously, can slow busy processors by consuming precious communication bandwidth.

It is expected that a parallel program using the shared-memory paradigm would contain several locks and critical sections. Since access to locks and corresponding critical sections is serialized, the amount of time spent by concurrent processes in waiting for the acquisition of locks can be a significant factor in the ultimate speedup achieved by the program. As the number of processes contending for a given lock increases, the average queuing delay per process is also bound to increase [14]. Further, it is intuitively clear that the length of a critical section would also influence the average queuing delay of processes in a parallel program. Since a longer critical section implies that the lock is busy for a longer period for each acquisition, the probability of new lock requests arriving during that interval is higher.

We isolate the overhead component that arises from waiting for software locks in the performance metric *queueing delay*. The overhead incurred from the execution of LOCK and UNLOCK routines (in the absence of any contention) is termed *software overhead*.

## 3. THE REPLICATE WORKLOAD FRAMEWORK

In this section, we describe the overall framework established for the performance measurement to ensure a fair comparison of results across different systems. Selection of the input parameters that characterize a particular kind of

workload and drive the performance measurement forms a very important and sensitive aspect of the study. Inclusion of too many parameters in the input set increases the complexity of the performance model and renders the interpretation of results more difficult. On the other hand, too few parameters in the input set could prove to be insufficient in accurately characterizing different kinds of realistic workloads, thus rendering the results meaningless. With this consideration in mind, we devote some verbage to explaining the input parameters in detail.

We have chosen, throughout our experiments, to keep the number of concurrently competing processes in our measurement framework less than the number of available processors. In this way, all processes can be simultaneously active on different processors and hence can be potential participants in the memory contention. If the number of processes is more than the number of processors, there is, in general, no hope for them all to be active simultaneously, and adding more processes cannot increase the contention. Thus, we confine our interest to the case of fewer processes than processors, or an equal number of each. In the rest of the paper, the terms process and processor are used interchangeably. Further, a process once created remains stationary on a processor and no process migration is allowed. This helps eliminate the context switch overhead that would entail from process migration. We also assume that other activity in the system during the measurement runs is negligible. This permits us to ascribe reasons with greater confidence to observed losses in efficiency.

### 3.1 Input Parameters and Workload Characterization

It is clear that the workload used in comparing the performance of different shared-memory architectures can have a strong influence on the results. For example, a (perhaps artificial) shared-memory workload exhibiting almost negligible shared-memory access will tend to favor a MIN-based distributed-memory architecture with memory local to each node over a bus-based architecture with no local cache. Moreover, measurement data on the behavior of real workloads are hard to come by. Because of this, we adopt a flexible model of workload characterization through variable input parameters. This model allows, through varying parameterizations of a single input set, the exploration of system performance over a wide spectrum of possible program behaviors.

The workload model is specified by six parameters:

$$\langle n, m, l, s, x, t \rangle.$$

The first parameter,  $n$ , specifies the number of concurrent processes to be created for the measurement run. As explained earlier, only one process is created per processor and

no migration is permitted. Thus  $n$  is always less than or equal to the maximum number of physical processors available on a system. This parameter controls the overall computational load on the system.

The parameter  $m$  specifies the number of shared-data objects that are used by the  $n$  concurrent processes. The  $m$  shared-data objects are evenly distributed across the available memory modules. For example, if  $m = 1$ , then a single data object is created in one of the memory modules selected at random, which is then accessed by all the processes. If  $m = n$ , then one data object is created in every memory module in the case of the BBNs, since there are  $n$  memory modules available when using  $n$  processors. On the Balance, however, the  $m$  shared-data objects are distributed evenly among the  $M$  (not necessarily equal to  $n$ ) available memory modules in an interleaved fashion. By setting the parameter  $m = 1$ , we effectively create a memory hot spot and hence can observe and quantify pure memory hardware contention.

The next three parameters together establish the personality of the program and hence define the workload. The parameter  $l$  defines the number of local computational operations (including local memory access) performed by each process between successive accesses to critical sections. This parameter controls the computational load of each processor. Similarly, the parameter  $s$  defines the number of shared-data references (not mutually exclusive) made by a process between successive accesses to critical sections. By setting the values of these two parameters to an appropriate ratio, different kinds of workloads may be selected.

The parameter  $x$  specifies the amount of time (in  $\mu s$ ) spent by a process within the critical section in each access to the critical section. In addition to the number of critical sections used in an application, the duration of each critical section is also crucial to the ultimate speedup. Varying this parameter controls the time for which locks are held by processes and hence allows a study of the lock contention characteristics of a given workload. Hence, it was chosen to specify  $x$  as an absolute time duration. On the other hand, specifying the parameters  $l$  and  $s$  in terms of absolute time demands a knowledge of the timing characteristics of the target system, which is not readily available. Moreover, it is the ratio of shared-data accesses to computational operations that is important and not their absolute durations. Hence,  $l$  and  $s$  are specified in terms of operations count.

The last parameter,  $t$ , indicates how the input parameters are to be interpreted in generating the memory reference patterns for the measurement run. Two types of models are provided—deterministic ( $t = 1$ ) and probabilistic ( $t = 0$ ). In the deterministic case, the values specified by  $l$ ,  $s$ , and  $x$  parameters are used as exact operation counts in generating the workload operational sequence. However, in the probabilistic case, the values specified by these parameters are used as mean values of randomly generated operation counts.

This permits the observation of the stochastic behavior of our model.

The *granularity* of a program, defined as the number of operations performed between synchronization points (like critical sections), is a key determinant in the interplay between a parallel application and an architecture. Similarly, the fraction of total operations devoted to shared-data access, called the *shared-access fraction*, controls the degree of interference among concurrent processes. For notational convenience, we define these two derived parameters in terms of the basic input parameters as follows:

$$\text{Granularity } \omega = l + s$$

$$\text{and Shared-Access Fraction } \rho = \frac{s}{l + s}.$$

To vary each of the input parameters independently over a wide range would yield a study so complex that it would be impossible to draw conclusions. Thus, in our study, we conduct controlled experiments using values for the parameters that are representative of some typical workloads as shown in Table II. It should be noted that the workloads selected for this study are not necessarily accurate representations of real parallel applications. Instead, a wide spectrum of workloads, some of them extreme, were deliberately used in the experimentation to illustrate the versatility of the representation scheme. However, that does not in any way detract from the flexibility of the scheme in its ability to model program behavior in practical applications. We also assume a homogeneous setting in which a single parameter value applies to all the processors.

Workload A forces traffic to a single memory module and hence measures the “hot-spot” performance of a system. Workload B, on the other hand, distributes memory references evenly among the memory modules and extracts the pure hardware contention of a system (no locks used in this case). Workload C measures the effect of granularity, shared-access fraction, and length of critical sections on the execution time of a parallel application as the number of concurrent processes is varied. Finally, Workload D determines the different worst-case overhead components under maxi-

TABLE II  
Parameter Settings for Different Workload Types

| Workload | $n$          | $m$     | $\omega$ | $\rho$     | $x$ ( $\mu s$ ) |
|----------|--------------|---------|----------|------------|-----------------|
| A        | 1 to $max^a$ | 1       | 100      | 0.0 to 0.4 | 0               |
| B        | 1 to $max$   | $m = n$ | 100      | 0.0 to 0.4 | 0               |
| C        | 1 to $max$   | $m = n$ | 100, 500 | 0.0 to 0.4 | 0 to 100        |
| D        | $max$        | $m = n$ | 500      | 0.1 to 0.4 | 0 to 150        |

<sup>a</sup> Depends on the maximum number of processors available on the system.

mal concurrency. Setting  $m = n$  in Workloads B, C, and D creates one shared-data object in each memory module in the case of the BBNs. The same setting is also used on the Balance only for the sake of uniformity in comparing the results obtained.

### 3.2. Program Structure

The program used for the performance measurement is written in C. It is structured so as to isolate the machine-dependent aspects and, hence, form a core measurement kernel that is independent of any particular shared-memory architecture. The operations that are machine dependent, like process forking, lock/unlock mechanisms, barrier synchronization, and time measurement, are encapsulated within machine-dependent macros that are selectively compiled into the code using the C preprocessor facility.

In order to represent various workload patterns, we need to have a generic program that can be adapted to represent different program behaviors. This program reads in each set of input parameters, emulates the execution of the specified workload, and generates a set of performance measurement results. To define different workloads, the proportion of local computations, shared-memory accesses, and critical section duration can be varied in the input parameter sets. The same workload is executed concurrently on  $n$  processors. The basic structure of the generic program executed by each processor appears in Fig. 1.

The sequence of operations performed by a processor from one access to a critical section to the next is called an *application cycle* or simply an "iteration." The number of iterations executed by each processor, *maxitr*, is fixed a priori at 10,000. Each iteration consists of  $\omega$  (granularity) operations followed by a critical section. For each operation, the pro-

cessor either performs a computation involving local memory access or generates a reference for a shared-data object in the proportion fixed by  $\rho$ . Each shared-data reference includes a *read* followed by a *write* to the shared-data location. This is done to force the reference to actually go out to shared memory even in the presence of data caching. The shared-data references are uniformly distributed over the  $m$  memory modules. At the end of the application cycle, the processor acquires a lock and enters a critical section for a time duration equal to  $x \mu s$ . If  $x = 0$ , then the lock acquisition and the critical section are bypassed in the cycle. The entire sequence of operations repeats once again for the next application cycle.

The sequence of operations for each process is generated using a random number generator, with the input parameters controlling the count for each type of operation. In order to minimize the intrusion due to program instrumentation on the actual measurement process, the complete operation sequence is generated outside the main performance measurement loop shown in Fig. 1. The initial 1% of the iterations are ignored to allow each process to reach steady state before the time measurements are taken.

The LOCK and UNLOCK routines are implemented by us using the low-level locking primitives provided in each system. LOCK uses spin waiting, but the process spins on a shadow copy of the global lock in local memory (or cache) until the lock becomes available. This prevents the interconnection network from being flooded with lock accesses and preserves bandwidth. UNLOCK enforces a first-come-first-served (FCFS) discipline on the use of the global lock by relinquishing the lock to the next process in the lock queue. In any case, at least one access to the global lock is required in the LOCK (UNLOCK) routines to set (reset) the lock.

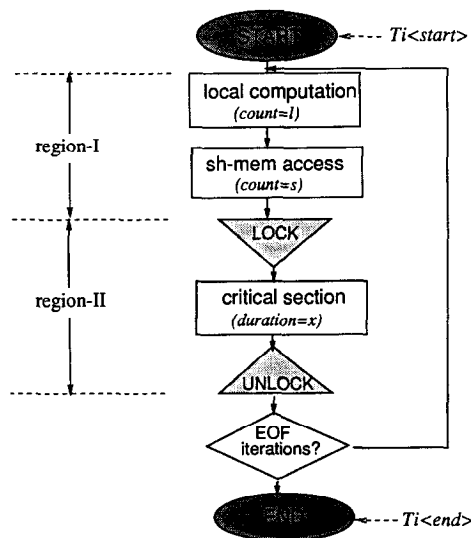


FIG. 1. Generic program structure for each process  $P_i$ .

### 3.3. Quantities Measured and Output Metrics

For each workload characterized by a set of input parameters, a corresponding set of performance measurement data that essentially consists of execution times is generated. However, since the processor speed and communication bandwidth of different multiprocessors are bound to be different, direct comparison of these execution times may not provide a meaningful insight into their relative performances. Furthermore, different systems support time measurement schemes with varying resolutions, which may influence the accuracy of the measured values. To provide a uniform basis for performance comparison, we need to normalize the execution times collected from each system. To this end, we introduce the output metrics *efficiency* and *overhead factor*.

We have described the term speedup, which is often employed as a metric for comparing performance of different architectures, for an application in Section 1. The important

fact to note in its definition is that the total amount of work done using  $n$  concurrent processors remains the same as that for a single processor; i.e., a sequential program is divided into smaller segments that are executed in parallel. However, in our performance model, instead of “decomposing” the total workload into smaller parallel segments, we “replicate” the workload by creating a concurrent instance of the program on each processor. Thus the total work done is scaled  $n$  times on  $n$  processors.

Now, referring to Fig. 1, let us define  $T_i$  as the *execution time per application cycle* for process  $P_i$ . If  $T_i\langle start \rangle$  and  $T_i\langle end \rangle$  denote the times measured at the start and end of a performance run, respectively, then

$$T_i = \frac{T_i\langle end \rangle - T_i\langle start \rangle}{maxitr}.$$

The execution times of the  $n$  concurrent processes are given by the  $n$ -tuple  $(T_1, T_2, T_3, \dots, T_n)$ . We define the *job execution time* on  $n$  processors,  $T(n)$ , as

$$T(n) = \max_{1 \leq i \leq n} \{T_i\}.$$

Obviously, in the single processor case  $T(1) = T_1$ . We are now ready to define efficiency, which provides a measure of the degree of degradation in performance of  $n$  concurrent processes due to contention for shared resources:

$$\text{Efficiency } \xi = \frac{T(1)}{T(n)}.$$

The efficiency  $\xi$  of running a program on a parallel architecture can also be interpreted as the ratio of the actual speedup achieved to the ideal speedup achievable on that architecture. Since the actual speedup achieved can never exceed the ideal speedup,  $\xi \leq 1$  always.

The loss in efficiency when  $n$  concurrent processes are executed can be attributed to several types of overhead as described in Section 2. To quantify this loss in performance, we define the term overhead factor as follows:

$$\text{Overhead Factor } \Theta = \frac{T(n) - T(1)}{T(1)}.$$

Note that  $T(1)$  is used in the denominator as a normalization entity; i.e., the overhead factor is expressed in units of the single-processor execution time. In order to examine the behavior of each overhead component, we factor out the contribution of each component in the total overhead as follows:

$O_l$  = overhead time due to network latency

$O_c$  = overhead time due to hardware contention

$O_q$  = overhead time due to lock contention

$O_s$  = overhead time of LOCK/UNLOCK routines.

We can now rewrite the expression for overhead factor  $\Theta$  as

$$\Theta = \frac{T(n) - T(1)}{T(1)} = \frac{O_l + O_c + O_q + O_s}{T(1)} = \theta_l + \theta_c + \theta_q + \theta_s,$$

which gives us the normalized overhead components  $\theta_l$  (*latency factor*),  $\theta_c$  (*contention factor*),  $\theta_q$  (*lock factor*), and  $\theta_s$  (*software factor*).

#### 4. A MODEL FOR OVERHEAD FACTORS

In this section, we formulate a mathematical model for the performance scenario described earlier. Before doing that, we introduce some basic cycle times that characterize the program execution on each system. All subsequent execution and overhead times are expressed in terms of these fundamental time units. Define

$t_p$  = average time per local computation operation (Fig. 1)

$t_a$  = average time per local memory access

$t_l$  = average latency per remote memory access

$t_c$  = average contention time per remote memory access

$t_{lk}$  = average time to execute the LOCK routine without contention

$t_{ul}$  = average time to execute the UNLOCK routine without contention.

The time  $t_a$  denotes the basic time required to access a local data object. The time  $t_l$  denotes the additional latency component incurred in accessing a remote data object. In the BBNs, the  $t_l$  component is nonzero since a remote memory reference goes out on the interconnection network whereas a local reference does not. Thus, in the absence of contention, the time for a remote memory access on the BBNs is given by  $t_a + t_l$ . However, in the Balance, the bus latency is subsumed in the basic memory access time  $t_a$  since it is an integral component of the memory access time. There is no additional delay incurred by “remote” references, since local and remote memories are indistinguishable, thus giving  $t_l = 0$ . The time  $t_c$  denotes the additional delay over and above the components  $t_a$  and  $t_l$  caused as a result of contention among concurrent memory accesses. Note that all the times defined above (except  $t_c$ ) are constant, being the characteristic of the underlying hardware/operating system, and do not depend on the workload. The values of  $t_{lk}$  and  $t_{ul}$  include the overhead of function call. A comparison of these fundamental unit times for the three systems under consideration is shown in Table III.

TABLE III  
Basic Time Measurements

| System  | $t_p$ ( $\mu$ s) | $t_a$ ( $\mu$ s) | $t_l$ ( $\mu$ s) | $t_{lk} + t_{ul}$ ( $\mu$ s) |
|---------|------------------|------------------|------------------|------------------------------|
| BBN-1   | 10.12            | 2.18             | 3.42             | 71.83                        |
| BBN-2   | 1.49             | 0.71             | 1.43             | 28.62                        |
| Balance | 37.22            | 10.85            | 0.00             | 83.18                        |

The remaining term,  $t_c$ , however, is dependent on the memory reference pattern and the communication bandwidth of the interconnection medium. It embodies the queuing delay experienced by a memory reference that must traverse the interconnection medium to be serviced. This delay arises from the interference between concurrent memory references at the destination memory module as well as on the network. We need to obtain an expression for  $t_c$  that reflects its dependence on the workload. Several earlier works have modeled memory interference for MINs using Markovian models [3] and probabilistic analysis [5, 15]. Similar work done for analyzing contention in bus-based systems includes [2, 6, 10].

#### 4.1. Contention Time on the BBNs

We use the result derived by Patel [15] using probabilistic analysis for Delta networks. The derived results apply to a  $p$ -stage MIN using  $k \times k$  switching elements. A memory access cycle (MAC) is defined as the time interval from the initiation of a memory request to the completion of the request. No distinction is made between read and write cycles in the analysis. The primary assumptions on which this analysis is based are as follows:

- (i) The memory references generated by each processor are independent of each other.
- (ii) The memory references are uniformly distributed over all the memory modules.
- (iii) All the  $k^p$  potential processors (since the system consists of a  $p$ -stage MIN with  $k \times k$  switches) in the system participate in the memory workload creation. If each processor generates memory requests at the rate of  $r$  requests per MAC, then for any input line of a switch in stage 1 of the MIN, the probability

$$\Pr[\text{a request arrives during a MAC}] = r.$$

The first two assumptions are satisfied by our performance measurement framework, where  $r$  is determined by the processor workload. However, since only  $n$  processors (out of the total capacity  $k^p$ ) participate in generating memory requests, the effective request rate at each stage 1 switch must be changed in assumption (iii). Assume that any processor

in the system could be selected to participate with equal probability. Now, for any switch input at stage 1, we have

$$\Pr[\text{input is active}] = n/k^p$$

$$\Pr[\text{a request arrives during a MAC} \mid \text{input is active}] = r$$

$$\Pr[\text{a request arrives during a MAC} \mid \text{input is not active}] = 0.$$

By using Baye's formula [8], we obtain

$$\Pr[\text{a request arrives during a MAC}] = \hat{r} = \left(\frac{n}{k^p}\right) \cdot r.$$

Thus,  $\hat{r}$  becomes the effective request rate at each input line of stage 1 switches. Using the new effective request rate, the probability  $P_A$  that an arbitrary memory request is accepted by the MIN (from [15]) is given by

$$P_A = \frac{r_p}{\hat{r}} = \frac{k^p}{n} \cdot \frac{r_p}{r}$$

where

$$r_i = 1 - \left(1 - \frac{r_{i-1}}{k}\right)^k \quad \text{and} \quad r_0 = \hat{r}. \quad (4.1)$$

We do not have a closed form solution for  $P_A$ , but plots [15] of  $P_A$  vs network size ( $k^p$ ) indicate that  $P_A$  decreases logarithmically as the network size increases.

On the BBN system, a memory conflict is essentially a conflict at the output line of a switch in the last stage of the MIN. Hence, it is accounted for by the switch contention analysis. The average number of wasted memory cycles per request,  $w$ , is easily computed by noting that a request that is rejected  $i$  times consecutively before being accepted waits for  $i$  cycles:

$$w = \sum_{i=0}^{\infty} i(1 - P_A)^i P_A = \frac{1 - P_A}{P_A} = \frac{n}{k^p} \cdot \frac{r}{r_p} - 1.$$

Hence, the average waiting time per request due to contention is

$$t_c = w(t_a + t_l) = \left(\frac{n}{k^p} \cdot \frac{r}{r_p} - 1\right)(t_a + t_l). \quad (4.2)$$

Independent, uniformly distributed references are not, however, an accurate model in the presence of global locks, even if all nonlock references are uniformly distributed.



Hence, the expression above will not apply accurately in situations with a single “spike” or hot spot in the memory reference pattern.

#### 4.2. Contention Time on the Balance

We use the result derived by Das and Bhuyan [6] using probabilistic analysis for multiple-bus multiprocessors. The derived results hold for a system with  $N$  processors,  $M$  memory modules, and  $B$  buses. We have adapted the expressions for the special case of a single-bus system ( $B = 1$ ) such as the Balance. Again, no distinction is made between read and write cycles as for the BBNs, and the analysis is based on the following assumptions:

- (i) The bus operation is synchronous; i.e., all requests are issued at the beginning of the bus cycle.
- (ii) The bus is circuit-switched; i.e., the bus is held for the entire duration of a memory access.
- (iii) The requests generated during a bus cycle are random and are independent of each other.
- (iv) The requests issued in successive cycles are independent of each other.

The fourth assumption is unrealistic because a rejected request will indeed be resubmitted in the next cycle. However, this assumption leads to simpler analysis, and it does not result in a substantial difference in the actual results [3]. Let  $r$  be the probability with which a processor generates a request in every bus cycle. The probability that there is at least one request for a memory module  $M_i$ , when  $n$  processors participate, is given (from [6]) by

$$X = 1 - \left(1 - \frac{r}{M}\right)^n.$$

The number of memory services requested in a cycle is a Binomial random variable with parameters  $M$  and  $X$ . Hence, the expected number of memory requests received per cycle is  $MX$ . Since only one of these requests can be accepted by the bus, the probability  $P_A$  that an arbitrary request is accepted can be written as  $P_A = 1/MX$ . Now, using an argument similar to that for the BBNs, the average number of wasted bus cycles per request,  $w$ , can be computed as

$$w = \frac{1 - P_A}{P_A} = (M - 1) - M \left(1 - \frac{r}{M}\right)^n.$$

Therefore, the average waiting time per request due to contention is

$$t_c = w t_{bus} = \left( (M - 1) - M \left(1 - \frac{r}{M}\right)^n \right) t_{bus}, \quad (4.3)$$

where  $t_{bus}$  is the bus cycle time (100 ns for the Balance). Note that all the terms in Eq. (4.3), except  $n$ , are constant for a given system and workload. The value of  $n$  varies according to the input parameters specified. It should also be noted that since successive memory requests on the Balance are pipelined onto the bus, the above equation provides only an upper bound for the contention time.

#### 4.3. Overhead Factors

Recall that  $T(1)$  was used as the unit of normalization in the definition of  $\Theta$ . We can express  $T(1)$  in terms of the workload parameters and the basic time units as

$$T(1) = l t_p + s t_a + x = \omega(1 - \rho) t_p + \omega \rho t_a + x.$$

Note that we have not added the terms ( $t_{lk} + t_{ul}$ ) in the above expression since an application with a single process does not need the service of a lock to exclusively access a shared resource. We can now express the overhead factors in terms of the time units defined earlier.

**Latency Factor.** For the Balance, since there is no distinction between local and remote memory, there is no additional overhead incurred by remote accesses. The bus latency constituent of the memory access time is subsumed in the basic memory access time  $t_a$ . Hence, there is no additional latency overhead, resulting in  $\theta_l = 0$ .

In the case of the BBNs, the latency overhead is contributed by those shared-data references that are sent out on the interconnection network. Every application cycle contains  $s$  references to shared data, each involving two accesses (read/write), and one shared access each for acquiring and releasing the lock. Converting the shared-memory reference count in terms of time, we obtain

$$\begin{aligned} \theta_l &= \frac{O_l}{T(1)} = \frac{1}{T(1)} (2s + 2) t_l \\ &= \frac{2(\omega \rho + 1) t_l}{\omega(1 - \rho) t_p + \omega \rho t_a + x}. \end{aligned} \quad (4.4)$$

**Contention Factor.** The contention overhead is contributed by all shared-data references (in both the BBNs and the Balance). Using the same arguments as that for latency factor above, we obtain the following result for contention factor for the BBNs as well as the Balance:

$$\begin{aligned} \theta_c &= \frac{O_c}{T(1)} = \frac{1}{T(1)} (2s + 2) t_c \\ &= \frac{2(\omega \rho + 1) t_c}{\omega(1 - \rho) t_p + \omega \rho t_a + x}. \end{aligned} \quad (4.5)$$

**Software Factor.** The pure software overhead arising out of a call to the LOCK and UNLOCK routines is a constant for a given system and a given implementation of these routines:

$$\theta_s = \frac{O_s}{T(1)} = \frac{(t_{lk} + t_{ul})}{\omega(1 - \rho)t_p + \omega\rho t_a + x}. \quad (4.6)$$

**Lock Factor.** This overhead arises from the contention for a global shared lock and the consequent queueing delay to acquire the lock. Let  $q$  denote the probability that at any instant of time, a process  $P_i$  is executing in region II of Fig. 1 (in the absence of any lock contention). Note that a process in region II could be in one of three possible states—waiting to acquire the lock, executing in the critical section, or trying to release the lock. We can express the probability  $q$  as the proportion of the iteration time spent by process  $P_i$  in region II:

$$q = \frac{x + 2t_c + (t_{lk} + t_{ul})}{\omega(1 - \rho)t_p + 2(\omega\rho + 1)(t_a + t_l + t_c) + x + (t_{lk} + t_{ul})}. \quad (4.7)$$

Since the workloads of all the concurrent processes in our model are identical, the probability  $q$  is the same for all of them. Now, let  $W$  be the number of processes already in region II when process  $P_i$  arrives at region II. It is clear that  $W$  is a Binomial random variable with parameters  $(n - 1)$  and  $q$ ; i.e.,  $W \sim B(n - 1, q)$ . Hence, the expected number of processes in region II when  $P_i$  arrives is given by

$$E[W] = (n - 1)q.$$

As the implementation of our locking protocol assigns the lock to processes in a FCFS fashion, the process  $P_i$  must wait for  $E[W]$  processes before it can acquire the lock. Thus, the average waiting time for the lock is given by

$$O_q = E[W] \cdot (x + t_c + t_{ul}).$$

We can now express the lock factor as

$$\theta_q = \frac{O_q}{T(1)} = \frac{(n - 1)q(x + t_c + t_{ul})}{\omega(1 - \rho)t_p + \omega\rho t_a + x}. \quad (4.8)$$

#### 4.4. Efficiency

From the definitions of  $\xi$  and  $\Theta$ , it can easily be verified that

$$\xi = \frac{1}{1 + \Theta} = \frac{1}{1 + (\theta_l + \theta_c + \theta_q + \theta_s)}, \quad (4.9)$$

which provides an indication of the trend that efficiency  $\xi$  will exhibit as the overhead factor  $\Theta$  varies.

### 5. DISCUSSION OF PERFORMANCE RESULTS

During the course of our experimentation, we tested more than 200 input parameter sets for both deterministic and probabilistic models. Each parameter set was constructed by varying the input parameters according to one of the workload forms shown in Table II. The range of parameters was selected with the goal of observing the sensitivity of  $\xi$  and  $\Theta$  to different types of workloads. The trends observed from data collected from the deterministic model were corroborated by data from the probabilistic model. As pointed out earlier, an attempt was made to ensure that the level of other activity in the system was negligible during our experimental runs (to minimize extraneous interference). We also tried to use the maximal number of processors on each system, within the constraints imposed by each site, to bring out the worst-case performance. The data presented in this section are excerpts from the probabilistic measurements. The results for some typical workloads are presented and discussed here.

#### 5.1. Workload A

This workload represents an extreme case in that it creates a hot-spot memory access pattern by forcing all processes to continually access a single memory module. The effect of a hot spot on the efficiency  $\xi$  is shown in Fig. 2 for two different values of shared-access fraction  $\rho$ .

As can be seen from Fig. 2, the efficiency drops by more than 50% at  $n = 20$ ,  $\rho = 0.1$  for the BBN systems. The performance degradation becomes even more pronounced as the values of  $n$  and  $\rho$  increase. On the other hand, the Balance is not affected as much by the hot spot, since the entire shared memory of Balance forms one indistinguishable unit. As long as the mean time between requests is greater than or equal to the memory access time  $t_a$ , there is no contention at the memory module and the Balance is able to service the requests efficiently.

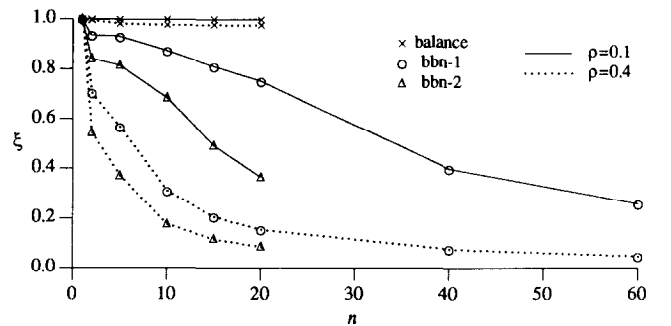
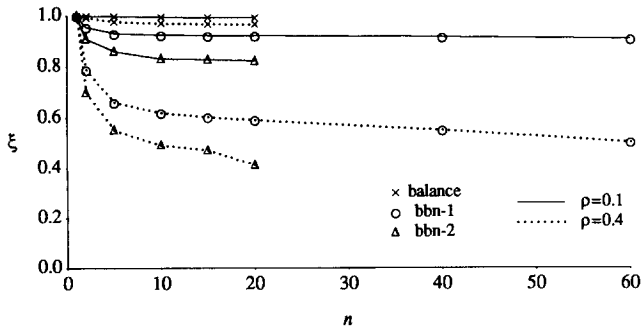


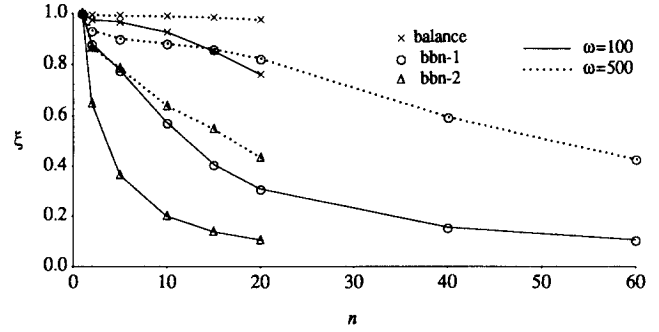
FIG. 2. Efficiency vs  $n$  ( $m = 1$ ,  $\omega = 100$ ,  $x = 0$ ).

FIG. 3. Efficiency vs  $n$  ( $m = n$ ,  $\omega = 100$ ,  $x = 0$ ).

Obviously, the deterioration in execution speed in the case of BBN-1 and BBN-2 is primarily due to the increasing contention overhead  $\theta_c$ . The expression for  $t_c$  (Eq. (4.2)), however, fails to explain the phenomenon as it is based on the premise that shared references are uniformly distributed over the memory modules. An explanation for the observed behavior is found in [16], a communication bandwidth analysis done for the RP3 system. It shows that the effective bandwidth of the network reduces drastically in the presence of a memory hot spot. This is true even when the fraction of total memory references directed at the hot spot is as low as 1%. The severe degradation in bandwidth occurs due to an effect called the "Tree Saturation Effect," which not only deteriorates the access time for the hot-spot references, but penalizes other references as well.

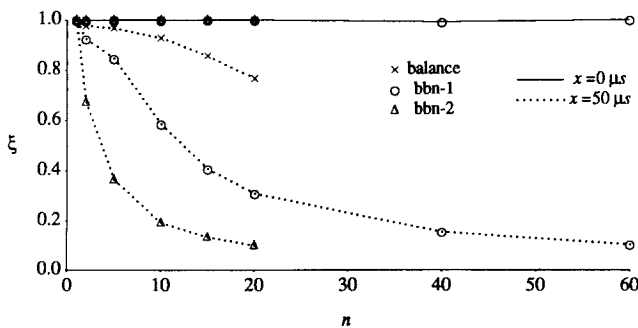
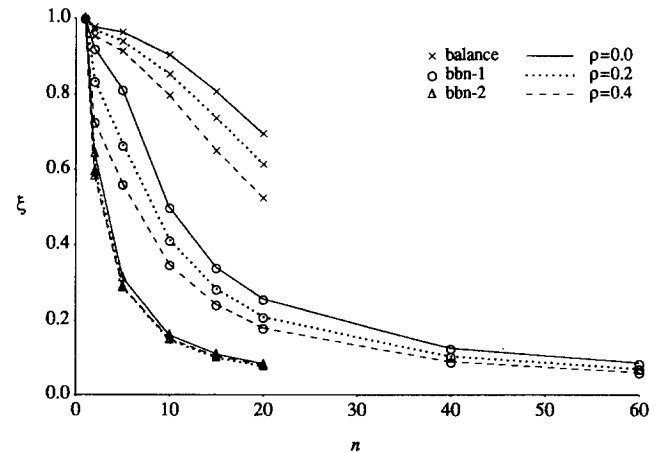
### 5.2. Workload B

This workload highlights the hardware overhead characteristics of each architecture. Figure 3 shows the trend in efficiency as the number of processors executing concurrently is varied. Since  $x = 0$  in this case, all the overhead is due to communication latency and contention in hardware. Clearly, both  $\theta_l$  and  $\theta_c$  depend on the shared-access fraction  $\rho$  and increase linearly with  $\rho$  as indicated by Eqs. (4.4) and (4.5). For the Balance,  $\theta_l = 0$  and, again, the loss in efficiency is small. Note that for the BBNs,  $\xi$  drops initially but remains

FIG. 5. Efficiency vs  $n$  ( $m = n$ ,  $\rho = 0.1$ ,  $x = 30 \mu s$ ).

relatively flat for higher values of  $n$ . This is due to the fact that as  $n$  increases, the number of memory modules also increases and the data references get redistributed uniformly over the memory modules. The efficiency curve for BBN-2 drops off faster than the corresponding curve for BBN-1. This can be inferred by examining the expression for  $t_c$  (Eq. (4.2)). The factor  $(n/k^p)$  in this equation signifies the fraction of the network capacity that is occupied. For a given value of  $n$ , this factor is larger for the BBN-2 ( $k = 8$ ,  $p = 2$ ) than for BBN-1 ( $k = 4$ ,  $p = 4$ ), thus yielding a larger value for  $t_c$  for BBN-2. However, the relatively flat shape of the curves for higher values of  $n$  points to the fact that the systems can be better utilized by using a larger number of processors to compensate for the loss in efficiency due to latency and contention.

Figure 4 illustrates the effect of introducing synchronization points into the program workload, where the synchronization occurs through a globally shared lock. Note that even in the absence of any other shared-data reference ( $\rho = 0$ ), the efficiency drops by more than 50% in the BBN-1 and the BBN-2. This, once again, vindicates the existence of the hot-spot problem on the BBNs, the shared lock being

FIG. 4. Efficiency vs  $n$  ( $m = n$ ,  $\omega = 100$ ,  $\rho = 0$ ).FIG. 6. Efficiency vs  $n$  ( $m = n$ ,  $\omega = 100$ ,  $x = 100 \mu s$ ).

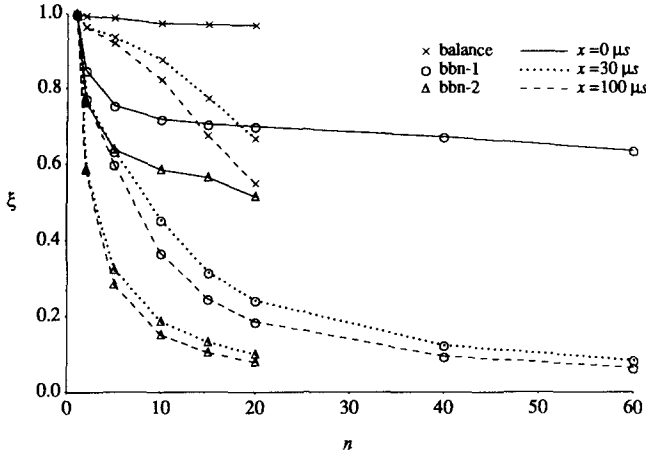


FIG. 7. Efficiency vs  $n$  ( $m = n$ ,  $\omega = 100$ ,  $\rho = 0.3$ ).

the hot-spot site in this case. The dominant contribution to total overhead was found to be from  $\theta_q$ . From Eq. (4.8), it can be seen that  $\theta_q$  increases linearly with  $n$ , but the bulk of the delay in the expression emanates from  $t_c$  for a hot-spot lock reference. The Balance, once again, does not suffer a significant loss in efficiency from the globally shared lock.

### 5.3. Workload C

The size of critical sections in parallel programs is usually kept small to alleviate the queueing delays at the critical

section entry points. Since critical sections introduce serialization bottlenecks into an otherwise parallel program, the granularity of the computation performed in parallel between these synchronization points must be appropriately selected to compensate for the synchronization overhead. Otherwise, the effective speedup gained from parallelization is sacrificed.

Figure 5 shows how efficiency is affected when the program granularity is changed from  $\omega = 100$  to  $\omega = 500$ . As can be seen from the graph, the efficiency improves for all the three systems when granularity is increased, keeping other parameters fixed. At  $n = 20$ , the increase in efficiency is approximately 24% for the Balance, 48% for the BBN-1, and 36% for the BBN-2. A key reason for this improvement can be ascribed to the fact that process executions get staggered in region I (Fig. 1), thus reducing the probability that the arrival of two processes at the critical section coincides. If one examines Eq. (4.7) for this probability  $q$ , it can be seen that an increase in  $\omega$  increases the denominator, thus yielding a smaller value of  $q$ . That, in turn, produces a smaller  $\theta_q$  in Eq. (4.8).

Figure 6 illustrates the loss in efficiency due to increased contention as  $n$  increases for three different values of shared-access fraction  $\rho$ . In the case of the Balance, increasing  $\rho$  leads to greater contention for the bus bandwidth, thus yielding a higher value of the contention factor  $\theta_c$ . Hence, a steady decrease in efficiency is observed as  $\rho$  is increased. In the BBNs, however, the additional deterioration in efficiency from increasing the value of  $\rho$  from zero to a positive quantity

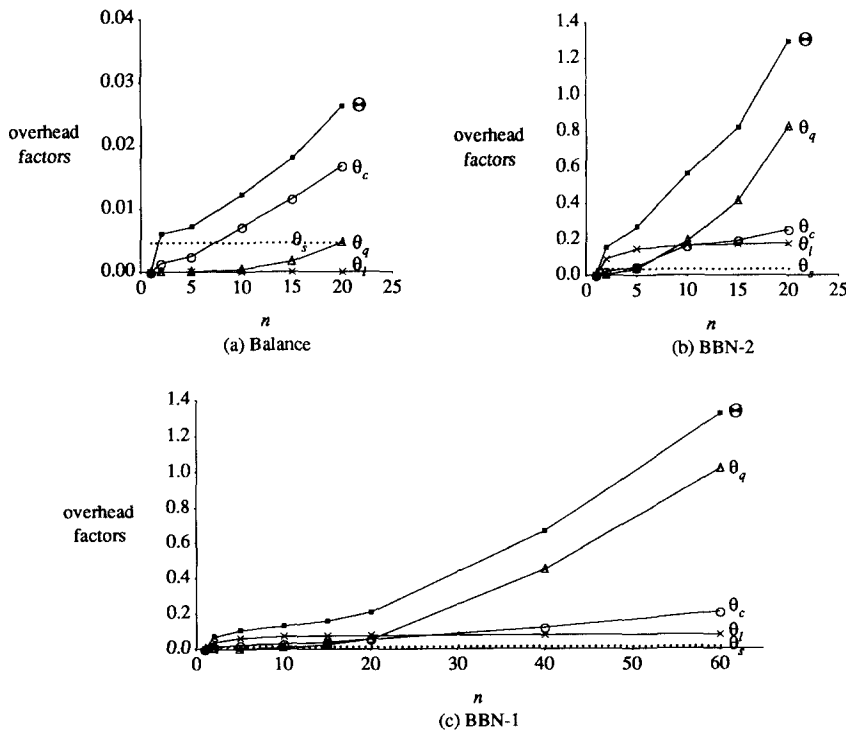
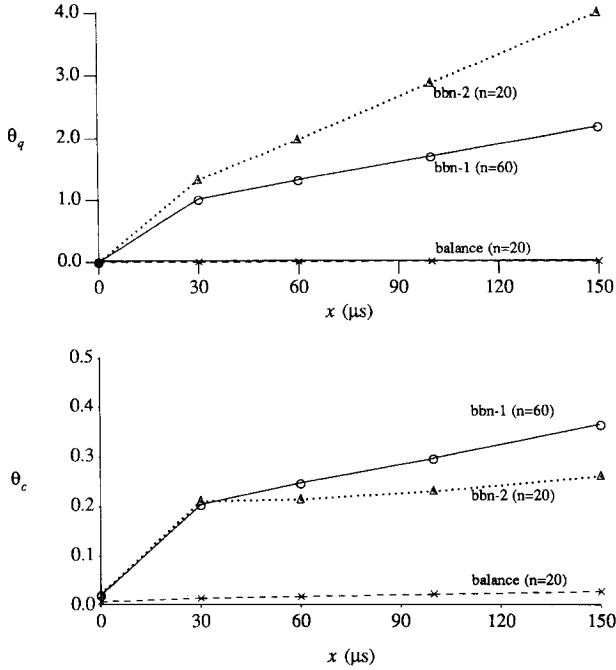


FIG. 8. Overhead components vs  $n$  ( $m = n$ ,  $\omega = 500$ ,  $\rho = 0.1$ ,  $x = 3 \mu s$ ).

FIG. 9. Overhead components vs  $x$  ( $m = n$ ,  $\omega = 500$ ,  $\rho = 0.1$ ).

is not so striking. This, once again, points to the fact that the performance degradation due to the shared-lock hot spot when  $\rho = 0.0$  still remains the dominant cause for overhead at  $\rho = 0.2$  and  $\rho = 0.4$ .

The influence of critical section length on the overall efficiency of a program workload is plotted in Fig. 7. The efficiency suffers on all three systems as the critical section length  $x$  is increased. Increasing the value of  $x$  results in a process having to wait for the shared lock for a longer time on the average, as indicated by Eq. (4.8). However, in the BBNs, the extent of loss in efficiency in going from  $x = 0$  to  $x = 30$  is far more significant than that in going from  $x = 30$  to  $x = 100$ . An increase in  $\theta_q$  proportional to  $x$ , as predicted by Eq. (4.8), does not explain this nonuniformity. The additional overhead, which causes this nonuniform behavior, is due to the introduction of a memory hot spot at the site of the shared lock for the critical section.

#### 5.4. Workload D

This workload was designed to study the effect of the input parameters on the individual overhead components. Figure 8 plots the individual overhead factors for the three systems as the degree of concurrency ( $n$ ) is varied under a fixed shared-access fraction ( $\rho$ ) and critical section length ( $x$ ). As explained earlier, the software overhead is a fixed and constant quantity. The latency factor  $\theta_l$  also remains fixed here as it depends only on the proportion of shared accesses  $\rho$ .

The  $\theta_c$  and  $\theta_q$  components increase steadily with  $n$  for all the three systems, as predicted by Eqs. (4.2) and (4.8).

For small critical section lengths, a process spends a greater proportion of its time in region I of Fig. 1 and, hence, the execution profile of the concurrent processes gets evenly distributed in region I. However, as  $x$  increases, the lock factor  $\theta_q$  begins to dominate as shown in Fig. 9. This is an outcome of the twofold effect that the critical section duration has on  $\theta_q$  in Eq. (4.8). An increase in the length of the critical section not only increases the  $x$  term, but also leads to an increase in the probability  $q$ . In fact, as the hardware technology gets faster (i.e.,  $t_p$ ,  $t_a$ , and  $t_l$  become smaller), the value of  $q$  increases even more for a given computational granularity  $\omega$  (Eq. (4.7)), further accentuating the  $\theta_q$  component. This fact is apparent from the  $\theta_q$  curve for BBN-2, which uses a faster technology. To compensate for the decrease in  $t_p$ ,  $t_a$ , and  $t_l$ , the computation granularity  $\omega$  must be increased to

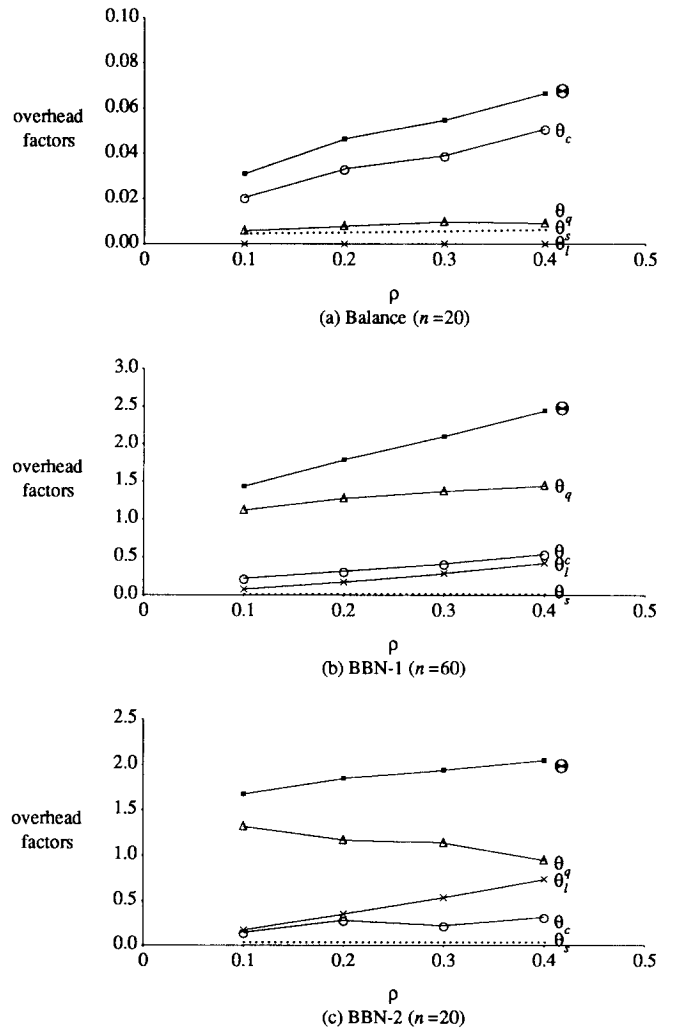
FIG. 10. Overhead components vs  $\rho$  ( $m = n$ ,  $\omega = 500$ ,  $x = 50 \mu s$ ).

TABLE IV  
Actual Execution Times ( $n = m$ ,  $\omega = 500$ ,  $x = 50 \mu s$ )

| Shared-access<br>fraction<br>$\rho$ | BBN-1 ( $n = 60$ )    |                       |       | BBN-2 ( $n = 20$ )    |                       |       | Balance ( $n = 20$ )  |                       |       |
|-------------------------------------|-----------------------|-----------------------|-------|-----------------------|-----------------------|-------|-----------------------|-----------------------|-------|
|                                     | $T(1)$<br>( $\mu s$ ) | $T(n)$<br>( $\mu s$ ) | $\xi$ | $T(1)$<br>( $\mu s$ ) | $T(n)$<br>( $\mu s$ ) | $\xi$ | $T(1)$<br>( $\mu s$ ) | $T(n)$<br>( $\mu s$ ) | $\xi$ |
| 0.1                                 | 4627.6                | 11,289.6              | 0.41  | 789.7                 | 2115.2                | 0.37  | 17,748.0              | 18,303.5              | 0.97  |
| 0.2                                 | 4208.9                | 11,756.3              | 0.36  | 770.8                 | 2199.8                | 0.35  | 16,086.2              | 16,834.0              | 0.95  |
| 0.3                                 | 3815.3                | 11,843.3              | 0.32  | 766.1                 | 2256.3                | 0.34  | 14,494.5              | 15,287.5              | 0.95  |
| 0.4                                 | 3455.3                | 11,906.2              | 0.29  | 744.9                 | 2273.7                | 0.33  | 12,984.4              | 13,849.3              | 0.94  |

prevent an increase in the value of  $q$ . On the Balance, the unit times  $t_p$  and  $t_a$  are very large, causing the term  $\omega(1 - \rho)t_p + 2(\omega\rho + 1)(t_a + t_c)$  in the denominator of Eq. (4.7) to overwhelm  $x$  in the range under consideration. This results in an extremely small value of  $q$ , thus making  $\theta_q$  negligible. The influence of  $x$  on  $\theta_c$  is only in as much as the creation of a hot-spot effect at the global lock on the BBNs.

Figure 10 shows the individual overhead components on the three systems as a function of the shared-access fraction  $\rho$ . Observe that the lock factor,  $\theta_q$ , is the largest overhead component on the BBNs, whereas the contention factor,  $\theta_c$ , is the largest on the Balance. The presence of a separate dedicated bus for shared-lock access in the Balance segregates the contention for lock access from those for other shared-memory access. Increased number of shared-memory accesses, as dictated by increasing  $\rho$ , leads to greater contention for the system bus bandwidth and a consequent increase in  $\theta_c$ . Changing the value of  $\rho$  also changes the fraction of time spent by a process in region I (Fig. 1). The exact nature of this change on any system is governed by the relative measures of  $t_p$  and  $t_a + t_l$  on that system. Also, note that the normalization factor  $T(1)$ , too, depends on the value of  $\rho$ . If  $t_p > t_a + t_l$ , then increasing  $\rho$  results in a smaller proportion of time in region I and a smaller value of  $T(1)$ . The results are just the opposite if  $t_p < t_a + t_l$ . Hence, the interpretation of the plots in Fig. 10 is closely related to the ratio of the

computational to memory access speeds of the individual systems.

All the performance figures presented so far have been normalized quantities. However, in order to provide a feel for the absolute speed of each system, we also enumerate some real execution times. Table IV shows the unnormalized values for execution times as  $\rho$  varies with a fixed parameter setting of  $n = m = 20$ ,  $\omega = 500$ , and  $x = 50 \mu s$ . It immediately reveals that the BBN-2 has the fastest and the Balance has the slowest execution times of the three systems. Table V documents the unnormalized overhead times corresponding to the same workload as that represented in Table IV. The software overhead time,  $O_s = t_{lk} + t_{ul}$ , is not included in this table. It is a fixed quantity for a given system and can be found from Table III.

## 6. CONCLUSIONS

A replicate workload model was developed to study the performance degradation in shared-memory multiprocessors due to shared-resource contention. In order to establish a standard basis for comparison of the relative loss in performance across systems with different architectures and speeds, two normalized output metrics were introduced—efficiency and overhead factor. The proposed model was employed to

TABLE V  
Actual Overhead Times ( $n = m$ ,  $\omega = 500$ ,  $x = 50 \mu s$ )

| Shared-access<br>fraction<br>$\rho$ | BBN-1 ( $n = 60$ )   |                      |                      | BBN-2 ( $n = 20$ )   |                      |                      | Balance ( $n = 20$ ) |                      |                      |
|-------------------------------------|----------------------|----------------------|----------------------|----------------------|----------------------|----------------------|----------------------|----------------------|----------------------|
|                                     | $O_l$<br>( $\mu s$ ) | $O_c$<br>( $\mu s$ ) | $O_q$<br>( $\mu s$ ) | $O_l$<br>( $\mu s$ ) | $O_c$<br>( $\mu s$ ) | $O_q$<br>( $\mu s$ ) | $O_l$<br>( $\mu s$ ) | $O_c$<br>( $\mu s$ ) | $O_q$<br>( $\mu s$ ) |
| 0.1                                 | 371.2                | 1040.4               | 5178.3               | 138.2                | 114.5                | 1043.2               | 0.0                  | 367.6                | 104.8                |
| 0.2                                 | 742.4                | 1357.2               | 5375.8               | 276.4                | 221.2                | 901.7                | 0.0                  | 537.4                | 127.3                |
| 0.3                                 | 1113.6               | 1605.7               | 5236.6               | 414.6                | 170.5                | 875.5                | 0.0                  | 567.9                | 142.3                |
| 0.4                                 | 1484.8               | 1888.9               | 5005.2               | 552.8                | 238.7                | 707.7                | 0.0                  | 661.9                | 119.8                |

measure and compare the performance of three contemporary multiprocessors, namely the BBN Butterfly-I, the BBN Butterfly-II, and the Sequent Balance. A special emphasis was placed on the BBN Butterfly-II as it is a relatively new system (still under Beta test) with no performance figures available to date.

The results obtained from the experimental runs showed that the interconnection network latency and switch contention were the largest proponents of overhead incurred on the BBN systems, and hence, the distribution of shared data in memory is of critical importance to the performance of a parallel application. On the Balance, however, the additional overhead due to bus contention was not significant due to the high bandwidth of the system bus.

In the presence of globally shared locks (critical sections), the queueing delay for the lock was found to be, by far, the single most significant source of overhead on the BBN systems. The severe loss in efficiency was attributed to the "hot-spot effect" for MINs. The Balance system was not subject to the hot-spot effect.

It should, however, be noted that on the Balance, the bandwidth is fixed by the bus at the outset. Adding more processors to the system leads to an increase in the contention for the bus, thus reducing its effective bandwidth. This imposes a severe restriction on the scalability of the system, limiting the maximum configuration of the system to about 30 processors. In contrast, the system bandwidth is expandable with the BBN system design. Since each processor module has its own memory and switch interface, adding a processor not only adds more memory, but also adds another communication path, increasing the total system bandwidth. Hence, the BBN systems maintain a balance among memory, processors, and bandwidth. The Butterfly switch also lets many different memory accesses occur simultaneously, which is not possible on bus-based systems.

Finally, the analytical formulation of our proposed model was used to corroborate the behaviors observed from the measured data.

## ACKNOWLEDGMENTS

The authors thank Mr. David Levine and Mr. Mark Henderson of Argonne National Laboratory for scheduling many of the experimental runs on the Sequent Balance system.

## REFERENCES

1. Abu-Sufah, W., and Kwok, A. Performance prediction tools for cedar: A multiprocessor supercomputer. *Proc. 12th International Symposium on Computer Architecture*, (1985), pp. 406-413.
2. Marsan, M. A., and Gerla, M. Markov models for multiple-bus multiprocessors. *IEEE Trans. Comput.* C-31 (Mar. 1982), 239-248.
3. Bhandarkar, D. P. Analysis of memory interference in multiprocessors. *IEEE Trans. Comput.* C-24 (Sept. 1975), 897-908.
4. Bhuyan, L. N. A combinatorial analysis of multibus multiprocessors. *Proc. 1984 International Conference on Parallel Processing*. IEEE Computer Society, Silver Spring, MD, Aug. 1984, pp. 225-227.
5. Bhuyan, L. N. An analysis of processor-memory interconnection networks. *IEEE Trans. Comput.* C-34 (Mar. 1985), 279-283.
6. Das, C. R., and Bhuyan, L. N. Bandwidth availability of multiplebus multiprocessors. *IEEE Trans. Comput.* C-34 (Oct. 1985), 918-926.
7. Dimpsey, R., and Iyer, R. Performance analysis of a shared-memory multiprocessor: Case study. *Proc. 1988 International Conference on Parallel Processing*. IEEE Computer Society, Silver Spring, MD, Aug. 1988.
8. Feller, W. *An Introduction to Probability Theory and Its Applications*. Wiley, New York, 1957, Vol. 1.
9. Feng, T. Y. A survey of interconnection networks. *IEEE Comput.* 14 (Dec. 1981), 12-27.
10. Fung, K. T., and Torng, H. C. On the analysis of memory conflicts and bus contentions in a multiple microprocessor system. *IEEE Trans. Comput.* C-27 (Jan. 1979), 28-37.
11. Heidelberg, P., and Trivedi, K. Queueing network models for parallel processing with asynchronous tasks. *IEEE Trans. Comput.* C-31 (1982), 1099-1108.
12. Herzog, U., Hoffman, W., and Kleinoder, W. Performance modeling and evaluation for hierarchically organized multiprocessor computer systems. *Proc. 1979 International Conference on Parallel Processing*. IEEE Computer Society, Silver Spring, MD, 1979, pp. 103-114.
13. Nanda, A. K., Shing, H., Tzen, T. H., and Ni, L. M. A replicate workload framework to study performance degradation in shared-memory multiprocessors. Tech. Rep. MSU-CPS-ACS-018, Department of Computer Science, Michigan State University, Jan. 1990.
14. Ni, L. M., and Wu, C. E. Design tradeoffs for process scheduling in shared memory multiprocessor systems. *IEEE Trans. Software Engrg.* 15 (Mar. 1989), 327-334.
15. Patel, J. H. Performance of processor-memory interconnections for multiprocessors. *IEEE Trans. Comput.* C-30 (Oct. 1981), 711-780.
16. Pfister, G. F., and Norton, V. A. Hot-spot contention and combining in multistage interconnection networks. *IEEE Trans. Comput.* C-34 (Oct. 1985), 943-948.
17. Vrsalovic, D., et al. Performance prediction for multiprocessor systems. *Proc. 1984 International Conference on Parallel Processing*. IEEE Computer Society, Silver Spring, MD, Aug. 1984, pp. 139-146.
18. *Balance 8000 System Technical Summary*. Sequent Computer Systems Inc., 1984.
19. *Overview of The Butterfly GP 1000*. BBN Advanced Computers Inc., Nov. 1988.
20. *TC2000 Technical Product Summary*. BBN Advanced Computers Inc., Nov. 1989.

---

ARUN NANDA received the B.E. degree in electrical and electronics engineering from BITS, Pilani, India, in 1984, and the M.S. degree in computer science from Michigan State University, East Lansing, in 1988. He is currently working toward his Ph.D. in the Department of Computer Science at Michigan State University, East Lansing. His research interests include multiprocessor performance evaluation, benchmarking, performance monitoring, and visualization. He is a member of the IEEE Computer Society and the Association for Computing Machinery.

HONDA SHING received the B.B.A. degree in computer science from Soochow University, Taipei, Taiwan, in 1983, and the M.S. degree in computer science from Michigan State University, East Lansing, in 1988. He is currently a Ph.D. candidate in the Department of Computer Science at Michigan State University. His research interests include parallel and distributed processing and shared-memory multiprocessor design. Mr. Shing is a member of the IEEE Computer Society.

TEN-HWAN TZEN received the B.S. degree in electrical engineering from National Tsing-Hua University, Hsinchu, Taiwan, in 1981, and the M.S. degree in computer science from Michigan State University, East Lansing, in 1988. He is presently a Ph.D. candidate in the Department of Computer Science at Michigan State University. His research interests are in the area of optimizing compilers and parallel and distributed processing.

LIONEL M. NI received the B.S. degree in electrical engineering from

National Taiwan University in 1973, the M.S. degree in electrical and computer engineering from Wayne State University, Detroit, in 1977, and the Ph.D. degree in electrical engineering from Purdue University, West Lafayette, Indiana, in 1980. He is currently a professor of computer science and the Director of the Advanced Computer Systems Laboratory at Michigan State University. During the summers of 1979 and 1981, he was a researcher at the IBM San Jose Research Laboratories. During the year of 1987–1988, he was a visiting scientist in the Division of Mathematics and Computer Science at Argonne National Laboratory. His current research interests include parallel processing and distributed computing. Dr. Ni has served as a distinguished visitor of the IEEE Computer Society from 1985 to 1988, the Program Chairman of the 15th IEEE Annual International Conference on Computer Software and Applications, and an associate editor of the *Journal of Parallel and Distributed Computing*. He is a senior member of the IEEE and a member of the Association for Computing Machinery and SIAM.

Received October 1990; revised March 28, 1991; accepted April 4, 1991